Developer note — WikiPortraits cards, v0.4.1

Audience: whoever maintains the other deployment of this tool.

Purpose: everything that changed between the version you have and v0.4.1, what you must change

with it, and how to check your port works.

Date: 2026-10-13. **App version:** `0.4.1` (`APP_VERSION` in `core.py`).

---

0. Which version do you have?

Check three things; any one of them tells you whether you need this note:

Where	Old	v0.4.1
`core.py` → `APP_VERSION`	`0.3.0`	`0.4.1`
`wiclient.py` → the questions dict	5 questions, no `publish_ready`	6 questions, includes `publish_ready`
Report heading for the ranked table	"Ranked by P(infobox-ready)"	"Ranked by composite"

The schema hash is a derived fingerprint of the questions:

old `f8b8a6b259ac`, new `cf59eead02a0`. It appears in cache file names, so you can also see

which answers on disk belong to which question set.

---

1. Breaking changes — act on these

1. **The question schema changed, so every cached answer is invalidated.** One question was added

(§2). The schema hash is part of the cache key, so all previously cached answers are simply

ignored and each photo costs **one fresh model call** the first time it is scored again. This is

deliberate, not a bug: answers to one question are only comparable within an identical question

set. Old `-f8b8a6b259ac-.json` files are inert; delete them or leave them.

2. **The ranking key changed.** The list is now ordered by `composite` (our arithmetic, §3.1), not by

`P(infobox_ready)`. Any UI copy, sort, threshold or downstream report that assumes "the headline

number is the model's infobox probability" is now wrong.

3. **CSV columns changed and shifted position.** New order:

`rank, title, pageid, status, composite, P_publish, P_infobox_ready, quality_0_4, pose,

P_eyes_to_camera, P_face_visible, input_tokens, wall_s, orig_px, sent_px, license, sha1, error`.

`composite` and `P_publish` were inserted after `status`, so **any positional parser breaks**.

4. **The** `/api/job/<id>` **payload gained keys**, all additive:

- per item: `composite`, `publish_p`, `thumb_file` (and `result.answers.publish_ready`);
- per job: `spread` (`composite` / `publish_ready` / `infobox_ready` sub-objects), `stats`

(now including `composite_mean`, `p_publish_mean`).

They are refreshed **as each photo settles**, not only when the run ends — a 24-photo run at

~57 s/call is 20+ minutes, and a finished photo must be readable long before the last one lands.

`job.pipeline` (the download/scoring overlap summary) is still written when the scoring phase

finishes, so a client that reads it should wait for `status == "done"`.

5. **The reports changed contents** (all four formats, §3.4). The routes and formats are unchanged.
6. **A writable directory is now required** for the thumbnail store (`cache/thumbs/`). If your platform has a read-only filesystem or an ephemeral disk that is wiped between requests, reports still work — they fall back to fetching the thumbnail from Commons when they are built — but they then need outbound access to `upload.wikimedia.org`/`thumb.wikimedia.org` at report time, and they will not be readable offline. Point `CACHE_DIR` at something writable and persistent if you can.
7. **Redeploy** `static/index.html` **together with the Python code.** The page reads the new payload fields (rank, composite bar, P(publish) bar, spread panel, extra About sections); old page + new server, or the reverse, gives a UI that silently shows nothing for the new numbers.

---

2. The model call

Six typed questions in one pass, no generated text (`output_tokens` is always 0). The added question, verbatim from `wiclient.py`:

```
publish_ready (choice)
  instructions: "An editor needs one photograph of this person."
  criteria:
    publish: "This photograph is the one to use."
    pass:    "Choose a different photograph instead."
```

Measured cost of the extra question (same photo, same 640 px width, n=1): input tokens **1,145 → 1,223** (+78, ≈ +7 %). Please re-measure on your own sample before budgeting; the earlier per-aspect-ratio table (`865 @16:9, 905 @3:2, 1,045 @1:1, 1,245 @2:3`) was for the five-question schema and is now stale.

---

3. New behaviour

3.1 The composite — our arithmetic, not the model's

`wiclient.composite(answers) = min(face_visible, eyes_to_camera) × pose weight`, where the pose weights are `head_shoulders 1.0 · three_quarter 0.9 · full_body 0.6 · candid 0.5 · group 0.25` (unknown pose → 0.5). Every ingredient is shown next to it so a reader can disagree with the weighting. **This must stay labelled as ours** in the UI, the About panel, the report method block and

your README — presenting it as a model probability would make the number unattributable, which is the

one thing this tool is built not to do. `P(publish)` and `P(infobox)` are the model's own answers and are always displayed beside it.

3.2 Candidate spread replaces standard deviation

`wiclient.candidate_spread(rows, floor)` reports, for each of the three headline numbers: candidates at or above the floor (`CANDIDATE_FLOOR = 0.25`), their range, the median gap between adjacent candidates, the dead-zone count and share (0.35–0.65) and the leading three titles.

**Range and median gap are `null` when there are fewer than two candidates — that is "undefined", not

zero**; a UI that prints `0.000` there is lying. Why not sd: sd rises whenever answers move toward

either end including the ceiling, so it rewards the exact failure we measured (rewording pushed every real portrait to 0.81–0.96 and shrank the range between them from 0.486 to as low as 0.021).

3.3 Per-photo values appear as photos settle

`core._derive()` / `core._refresh_derived()` set `composite`, `publish_p` and the run `spread` at every settle, and are called from both branches (cache hit and fresh score). Previously these were computed only after the last photo, so a long run displayed nothing at all until it was over.

3.4 Reports now show the photo beside its evaluations

- **Markdown**: the thumbnail inline above each photo's determinations, plus a new `- Licence: … · photo: …` line (or "see the Commons file page" when the API returned none).
- **PDF**: the photo embedded as a real image — JPEG passed through as `/DCTDecode`, PNG decoded to raw RGB and embedded as `/FlateDecode` (palette, greyscale and alpha handled; alpha is composited onto white). Pure stdlib (`zlib`, `struct`), no PIL. **Measured: 0.31 s to decode a 960×1193 PNG**, and the decoded RGB for that image is 3.4 MB in memory — that is the one place memory scales with
image size, so keep batches modest on a small container. Every embedded image gets a hairline frame:
white-on-transparent artwork is otherwise invisible on white paper and the reader cannot tell an image was embedded.
- Thumbnails are saved to `cache/thumbs/` at fetch time, so the PDF embeds **the exact bytes sent to the model** and stays readable offline. A cached answer with no saved thumbnail is backfilled from Commons once (`core.saved_thumb()` → fallback fetch).
- A photo that cannot be embedded is listed at the end of the PDF under "Images that could not be embedded" instead of being dropped silently.
- **Attribution:** because reports embed and therefore redistribute Commons media, the licence and photographer now sit with the photo. If you serve reports publicly, keep that line.

---

4. Robustness fixes (three real bugs)

1. **A missing import killed the worker thread.** `thumb_path()` called `re.sub` with no `import re` in `core.py`. The exception killed the download producer thread, and the pipeline consumer then blocks forever on `q.get()` — which looks exactly like the model service hanging. **Diagnosis rule: a run stuck at "n-1 of n" is a dead thread until proven otherwise — grep the log for `Exception in thread` before blaming latency.** (This cost me a wrong conclusion: I first reported the endpoint as degraded to >25 min/call. It was this bug.)
2. **A bare `except Exception` hid it.** The thumbnail-saving helper swallowed the `NameError` and silently degraded to "no image in the report", which let the offline test suite pass. Worker-level handlers now catch `OSError` only, and `test_pipeline.py` asserts that every scored photo has a saved thumbnail, so this class of regression fails the suite instead of shipping.
3. **Near-empty trailing page** in the PDF: the closing disclaimer forced a page break. The writer now checks remaining height (`pdfmin.remaining_h()`) and drops the spacer when the page is tight.

---

5. Verifying your port

Run the suites; all of these must be clean:

```
bash run_all_tests.sh            # offline suites + the live contract test
python3 test_reports.py          # composite, P(publish), spread maths, MD/PDF images, licence lines
python3 check_js.py              # the inline page script parses (needs node)
node test_render_dom.js          # results list, ranking, spread panel in a DOM (needs npm install)
python3 test_ui_contract.py URL  # UI wiring, payload fields, About content against a running server
python3 verify_live_two.py URL   # one cached + one fresh photo end to end, all four reports
```

Then check these specific things on your deployment, since they are environment-sensitive:

- **Payload:** after one photo settles, `items[0].composite`, `items[0].publish_p` and `job.spread.composite.n` are all non-null **while** the run is still going.
- **Cache invalidation:** photos scored under the old schema re-score once; the second run of the same

photo reports `from_cache: true` and costs nothing.

- **Report images:** `GET /api/job/<id>/report.pdf` contains at least one `/Subtype /Image` object per scored photo, and `report.md` contains one `!…` per scored photo plus a `Licence:` line.
- **Writable thumbnail store:** after a fresh score, `cache/thumbs/*.img` exists; if it does not, check the platform's filesystem permissions and confirm the Commons fallback still produces a PDF with images.
- **About panel** still matches the code that scores: the questions table must list six questions and the panel must state that the composite is not a model probability.

---

6. Tests and files added or changed

New: `test_reports.py`, `check_js.py`, `verify_live_two.py`, `verify_reports_live.py`, `diag_contract_job.py`, `test_pdf_images.py`, `research/sample-report.{pdf,md}` (+ a rendered page PNG), `research/wording-experiment-2026-10-13.md` and its raw data.

Changed: `core.py`, `wiclient.py`, `reports.py`, `pdfmin.py`, `static/index.html`, `test_pipeline.py`, `test_render_dom.js`, `test_ui_contract.py`, `run_all_tests.sh`, `package.sh`, `PRD.md`, `README.md`, `HANDOFF.md`.

No new dependencies. Still stdlib-only (the PNG work uses `zlib` and `struct`), which is what keeps the Toolforge/WSGI path working without a build step.

---

7. Known limitations — please do not oversell these

- The composite's **pose weights are hand-chosen, not fitted**, and the composite has not been validated against human labels. It is a ranking device, not a measurement.
- The validation run that would settle this (12+ portraits, composite vs P(publish) vs P(infobox), then hand labels) has **not been done**.
- Percentile-within-run is still not built; only the rank is.
- Category input, run persistence/comparison and a calibrated threshold are **not shipped**.
- Endpoint latency is currently ~57 s/call, ≈105× the measured 0.54 s warm baseline, entirely service-side (HTTP 200, no 429s) — it is not caused by the app, the image size or the prompt size,

and it is worth reporting to the LiftWing maintainers.

- A one-photo PDF can still spill its closing disclaimer onto a short second page.

---

8. Rollback

Reverting `wiclient.py`, `core.py`, `reports.py`, `pdfmin.py` and `static/index.html` to your current versions restores the previous behaviour completely. Two things to know: the schema hash is derived from the questions, so a reverted `wiclient.py` restores the old hash and makes any old `-f8b8a6b259ac-.json` cache entries valid again (if you kept them); and the thumbnail store under `cache/thumbs/` is additive, so it can be left in place or deleted safely.